Arquitectura por capas: MVC + DAO + Service Layer

Contenido

Introducción 2
1. Capa Modelo — Model 2
2. Capa Vista — View 2
3. Capa Controlador — Controller 2
4. Capa de Servicios — Service Layer 2
5. Capa de Acceso a Datos — DAO 3
6. Base de datos 3
Ejemplo de aplicación típica: 5
1. Model — Modelo 5
2. View — Vista 6
3. Controller — Controlador 8
4. Service Layer — Capa de servicios 9
5. DAO — Data Access Object 10
6. ¿Por qué separar Service y DAO? 11
7. Flujo completo de una operación 12
8. ¿Dónde encaja MVC? 14
9. Ejemplo de estructura de un proyecto Java 15
10. Ventajas de esta arquitectura 16

Introducción

La **arquitectura por capas** consiste en dividir una aplicación en diferentes niveles, de manera que cada uno tenga una **responsabilidad concreta**. El objetivo principal es conseguir un código más **organizado, mantenible, reutilizable y fácil de probar**.

1. Capa Modelo — Model

Es la capa encargada de representar los datos, entidades y conceptos del dominio de la aplicación, definiendo su estructura y, cuando corresponda, las reglas propias de dichos objetos.

Su función es representar **qué información maneja la aplicación**.

---

2. Capa Vista — View

Es la capa responsable de la presentación de la información al usuario y de la interacción con este, mostrando los resultados de las operaciones y recogiendo sus acciones o datos de entrada.

Su función es gestionar **cómo se presenta la información al usuario**.

---

3. Capa Controlador — Controller

Es la capa encargada de recibir y gestionar las peticiones o acciones procedentes de la vista, coordinando el flujo de la aplicación y delegando la ejecución de la lógica de negocio en la capa de servicios.

Su función es **coordinar las peticiones entre la vista y el resto de capas**.

---

4. Capa de Servicios — Service Layer

Es la capa que encapsula la lógica de negocio de la aplicación, implementando las operaciones y reglas que determinan cómo debe comportarse el sistema, y coordinando, cuando sea necesario, el acceso a los datos mediante los objetos DAO.

Su función es determinar **qué debe hacer la aplicación y bajo qué reglas**.

---

5. Capa de Acceso a Datos — DAO

Es la capa encargada de encapsular y abstraer las operaciones de acceso y persistencia de datos, proporcionando una interfaz para consultar, insertar, modificar y eliminar (CRUD es un acrónimo de Create, Read, Update y Delete) información en la fuente de datos.

Su función es gestionar **cómo se almacenan y recuperan los datos**, aislando al resto de la aplicación de los detalles de la base de datos.

---

6. Base de datos

Es el sistema encargado de almacenar de forma persistente la información utilizada por la aplicación y de proporcionar los mecanismos necesarios para su consulta y modificación.

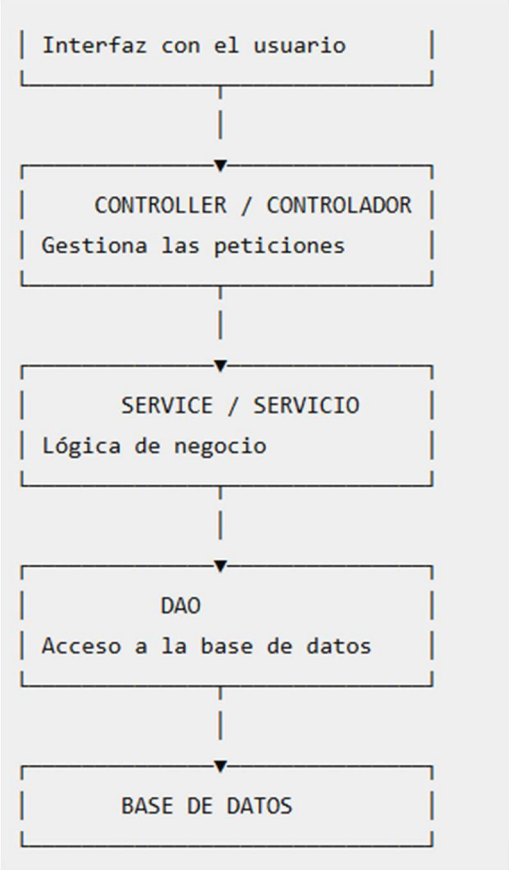
Su función es **persistir los datos**.

Resumen formal

Capa	Definición resumida
Model	Representa los datos y conceptos del dominio.
View	Presenta la información e interactúa con el usuario.
Controller	Recibe peticiones y coordina el flujo de la aplicación.
Service	Contiene y ejecuta la lógica de negocio.
DAO	Encapsula el acceso y la persistencia de datos.
Base de datos	Almacena persistentemente la información.

Regla para recordar:
Vista → Controlador → Servicio → DAO → Base de datos.

Ejemplo de aplicación típica:



El **Model** de MVC aparece transversalmente, ya que representa los datos que maneja la aplicación.

---

1. Model — Modelo

El **Modelo** representa los **datos y objetos del dominio de la aplicación**.

Por ejemplo, en una aplicación de gestión de alumnos podríamos tener:

```
public class Alumno {
    private int id;
    private String nombre;
    private String email;

    // constructores, getters y setters
}
```

La clase Alumno representa una entidad de nuestro sistema.

¿Qué contiene?

Habitualmente:

- Entidades.
- Clases que representan los datos.
- Atributos.
- Constructores.
- Getters y setters.
- En algunos proyectos, validaciones relacionadas con los propios datos.

Por ejemplo:

```
model/
  ├── Alumno.java
  ├── Profesor.java
  └── Curso.java
```

¿Qué NO debería hacer?

El modelo no debería encargarse directamente de:

- Mostrar información por pantalla.
- Recibir peticiones HTTP.
- Ejecutar consultas SQL.
- Gestionar la interfaz gráfica.

Su responsabilidad principal es **representar los datos y conceptos del sistema**.

---

2. View — Vista

La **Vista** es la parte de la aplicación que permite **mostrar información al usuario y recoger sus acciones**.

Dependiendo de la tecnología puede ser:

- HTML/CSS/JavaScript.
- JSP.
- JavaFX.
- Swing.
- Thymeleaf.
- Una aplicación móvil, etc.

Por ejemplo:

```
Alumno: Juan García
Email: juan@email.com

[Guardar] [Cancelar]
```

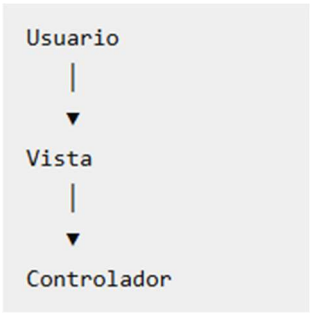
La vista debe centrarse principalmente en la **presentación de la información**.

Responsabilidad

La vista:

1. Muestra información.
2. Recoge datos introducidos por el usuario.
3. Envía las acciones al controlador.
4. Muestra el resultado recibido del controlador.

Ejemplo



La vista **no debería acceder directamente a la base de datos**.

3. Controller — Controlador

El **Controlador** actúa como intermediario entre la vista y el resto de la aplicación.

Su función principal es **recibir las acciones o peticiones del usuario y decidir qué operación debe realizarse**.

Por ejemplo:

```
public void guardarAlumno(Alumno alumno) {
    alumnoService.guardar(alumno);
}
```

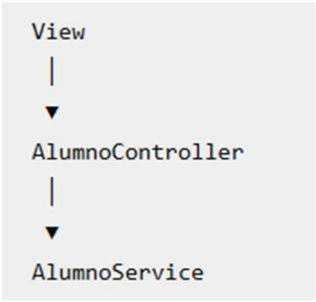
El controlador recibe la petición y delega el trabajo en la capa de servicios.

Ejemplo de flujo

Un usuario pulsa:

[Guardar alumno]

La vista envía la petición:



El controlador **no debería contener toda la lógica de negocio**.

Por ejemplo, no sería recomendable hacer esto:

```
public void guardarAlumno(Alumno alumno) {

    // Validaciones
    // Cálculos
    // SQL
    // Gestión de transacciones
    // etc.
}
```

El controlador debe ser relativamente **ligero** y delegar responsabilidades.

---

4. Service Layer — Capa de servicios

Esta capa contiene la **lógica de negocio de la aplicación**.

Es una de las capas más importantes de la arquitectura.

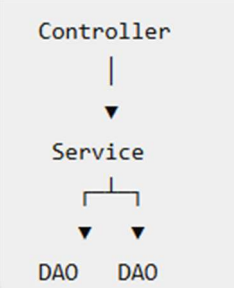
Aquí se decide **qué debe hacer realmente el sistema**.

Por ejemplo:

```
public void matricularAlumno(int alumnoId, int cursoId) {

    // Comprobar que existe el alumno
    // Comprobar que existe el curso
    // Comprobar que puede matricularse
    // Realizar la matrícula
}
```

La capa Service puede utilizar uno o varios DAO.



Ejemplo

Supongamos que queremos realizar una transferencia bancaria:

```
transferir(cuentaOrigen, cuentaDestino, cantidad);
```

La lógica podría ser:

1. Comprobar que las cuentas existen.
2. Comprobar que la cantidad es válida.
3. Comprobar que hay saldo suficiente.

4. Restar dinero de una cuenta.
5. Añadir dinero a la otra.
6. Confirmar la operación.

Esto es **lógica de negocio**, por lo que pertenece al Service.

---

5. DAO — Data Access Object

DAO significa:

Data Access Object

Su función es **encapsular el acceso a los datos**, normalmente a una base de datos.

Por ejemplo:

```
public interface AlumnoDAO {

    Alumno buscarPorId(int id);

    List<Alumno> buscarTodos();

    void insertar(Alumno alumno);

    void actualizar(Alumno alumno);

    void eliminar(int id);
}
```

Después podemos tener una implementación:

```java
public class AlumnoDAOImpl implements AlumnoDAO {

    @Override
    public Alumno buscarPorId(int id) {
        // SELECT ...
    }

    @Override
    public void insertar(Alumno alumno) {
        // INSERT ...
    }
}
```

¿Qué hace el DAO?

Se ocupa de operaciones como:

```
SELECT
INSERT
UPDATE
DELETE
```

y de la comunicación con:

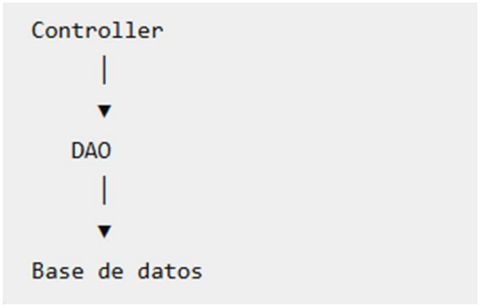
- PostgreSQL.
- MySQL.
- Oracle.
- SQLite.
- etc.

---

6. ¿Por qué separar Service y DAO?

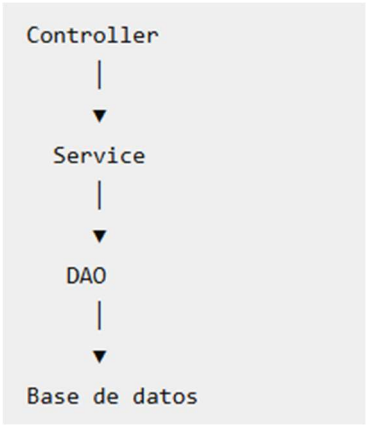
Esta separación es especialmente importante.

Imaginemos:



El problema es que la lógica de negocio acabaría mezclada con las consultas SQL.

Con Service Layer:



Tenemos responsabilidades claramente separadas:

Capa	Responsabilidad
View	Presentación
Controller	Gestionar peticiones
Service	Lógica de negocio
DAO	Acceso a datos
Model	Representar los datos
BD	Almacenar los datos

7. Flujo completo de una operación

Supongamos que el usuario quiere **crear un alumno**.

Paso 1 — Vista

```
Nombre: Ana López
Email: ana@email.com

[Guardar]
```

La vista envía la información al controlador.

Paso 2 — Controller

```
alumnoController.crear(alumno);
```

El controlador delega en el servicio:

```
alumnoService.crear(alumno);
```

Paso 3 — Service

El servicio realiza las comprobaciones necesarias:

```
public void crear(Alumno alumno) {

    validarAlumno(alumno);

    if (alumnoDAO.existeEmail(alumno.getEmail())) {
        throw new IllegalArgumentException(
            "El email ya existe"
        );
    }

    alumnoDAO.insertar(alumno);
}
```

}

Paso 4 — DAO

El DAO se encarga de hablar con la base de datos:

```
String sql =
    "INSERT INTO alumno(nombre, email) VALUES (?, ?)";
```

Paso 5 — Base de datos

Finalmente se almacena:

```
ALUMNO
--------------------------------
id | nombre     | email
--------------------------------
1  | Ana López  | ana@email.com
```

Y la respuesta vuelve:

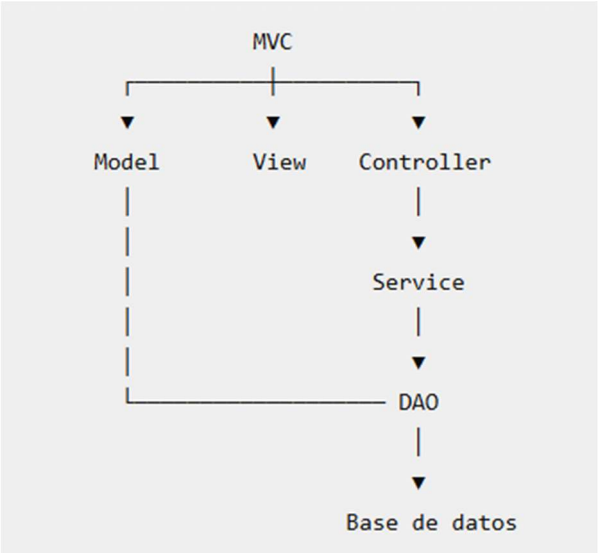
```
BD
 ↓
DAO
 ↓
Service
 ↓
Controller
 ↓
View
 ↓
Usuario
```

8. ¿Dónde encaja MVC?

MVC divide principalmente la aplicación en:

```
M → Model
V → View
C → Controller
```

Mientras que DAO y Service Layer añaden una separación más profunda:



Por eso es frecuente encontrar proyectos que utilizan **MVC + Service + DAO** conjuntamente.

No son patrones que necesariamente compitan entre sí: **resuelven problemas diferentes y pueden combinarse**.

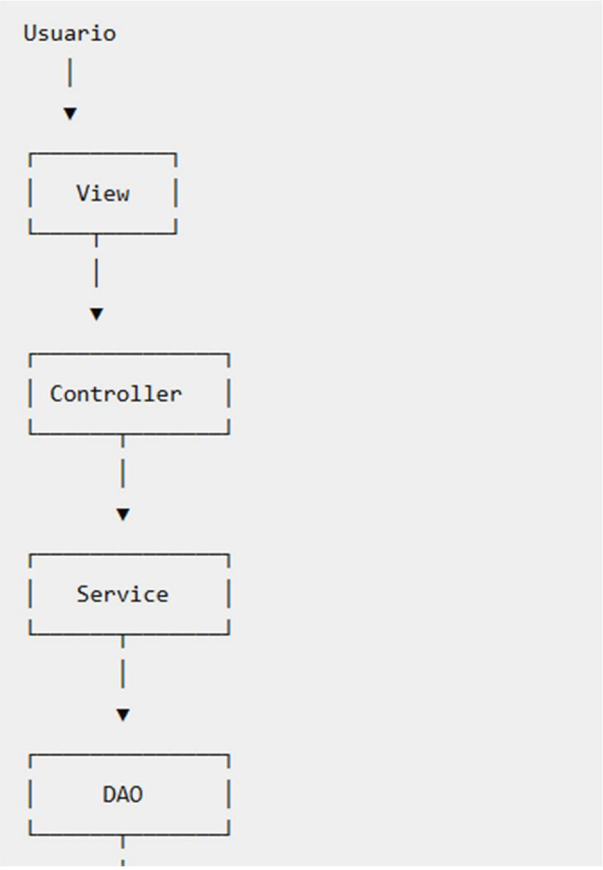
---

9. Ejemplo de estructura de un proyecto Java

Una posible organización sería:

```
└── main/
    └── java/
        └── com.ejemplo/
            │
            ├── controller/
            │   └── AlumnoController.java
            │
            ├── model/
            │   └── Alumno.java
            │
            ├── service/
            │   ├── AlumnoService.java
            │   └── AlumnoServiceImpl.java
            │
            ├── dao/
            │   ├── AlumnoDAO.java
            │   └── AlumnoDAOImpl.java
            │
            └── util/
                └── ConexionBD.java
```

Una petición típica seguiría este camino:



10. Ventajas de esta arquitectura

Separación de responsabilidades

Cada clase tiene una función concreta.

Mantenimiento

Si cambia la base de datos, podemos modificar principalmente el DAO sin tener que modificar toda la aplicación.

Reutilización

Un mismo AlumnoService puede ser utilizado por diferentes controladores.

Testabilidad

Podemos probar cada capa independientemente.

Escalabilidad

Cuando el proyecto crece, es más fácil incorporar nuevas funcionalidades sin crear clases excesivamente grandes.

Idea fundamental para el alumnado

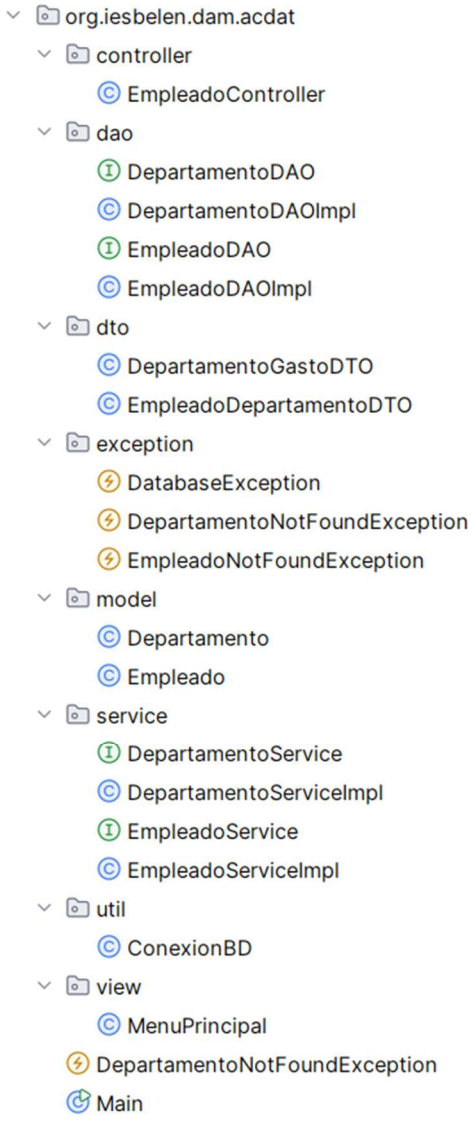
Una forma sencilla de recordar qué hace cada capa es:

View: ¿Cómo se lo muestro al usuario?
Controller: ¿Qué ha solicitado el usuario?
Service: ¿Qué tiene que hacer realmente la aplicación?
DAO: ¿Cómo guardo o recupero los datos?
Model: ¿Qué datos maneja la aplicación?

Y, sobre todo:

```
VIEW
  ↓
CONTROLLER
  ↓
SERVICE
  ↓
DAO
  ↓
BASE DE DATOS
```

La petición baja por las capas y el resultado vuelve hacia arriba.


org.iesbelen.dam.acdat
controller
EmpleadoController
dao
DepartamentoDAO
DepartamentoDAOImpl
EmpleadoDAO
EmpleadoDAOImpl
dto
DepartamentoGastoDTO
EmpleadoDepartamentoDTO
exception
DatabaseException
DepartamentoNotFoundException
EmpleadoNotFoundException
model
Departamento
Empleado
service
DepartamentoService
DepartamentoServiceImpl
EmpleadoService
EmpleadoServiceImpl
util
ConexionBD
view
MenuPrincipal
DepartamentoNotFoundException
Main